1. Project Proposal

Project Topic: Case study 3: Citizen Sourcing-based System for Smart City and Smart Governance of Pathumthani (CSSS)

This project designs the database to keep data that people in Pathumthani Province report their problem to the government of Pathumthani by using SQL to create. The case study will show information about citizens, types of issues, offices for each department, location of issue, issue, feedback from citizens, and social media.

The data and the relationship for each table:

- **Citizen information**

Citizen ID, full name, phone number, email, and the date that they reported the issue.

It is also important to note that every report is stored as a unique issue, even if several citizens are reporting the same real-world event. The responsibility to group or merge such reports lies with the authorities, not the database. This ensures that no citizen's voice is lost and every complaint is tracked individually.

- **Types of issues**

There are 8 types including category ID,transportations, utilities, waste, crimes, health, accident, environment, and disaster.

- **Offices for each department**

The departments depend on the types of issues. There are the transportations department, utilities department, waste department, crimes department, health department, accident department, environment department, and disaster department. Moreover, there is an office ID to specify the departments.

- **Location of issue**

The location of issues can be updated for all the time according to the issue occurrence. Therefore, the location of issues has a relationship with the issue topic. The location will show location ID (it will link with the issue), latitude, longitude, district, subdistrict, postcode, street, and building number.

- **Issue**

This data will link from almost all topics. The data includes issue ID, category ID (link category ID from types of issues), office ID (link office ID from offices), location ID (link location ID from location of issue), description (to specify the citizen problem), current status (to see the progress), reported date, started date and completed date.

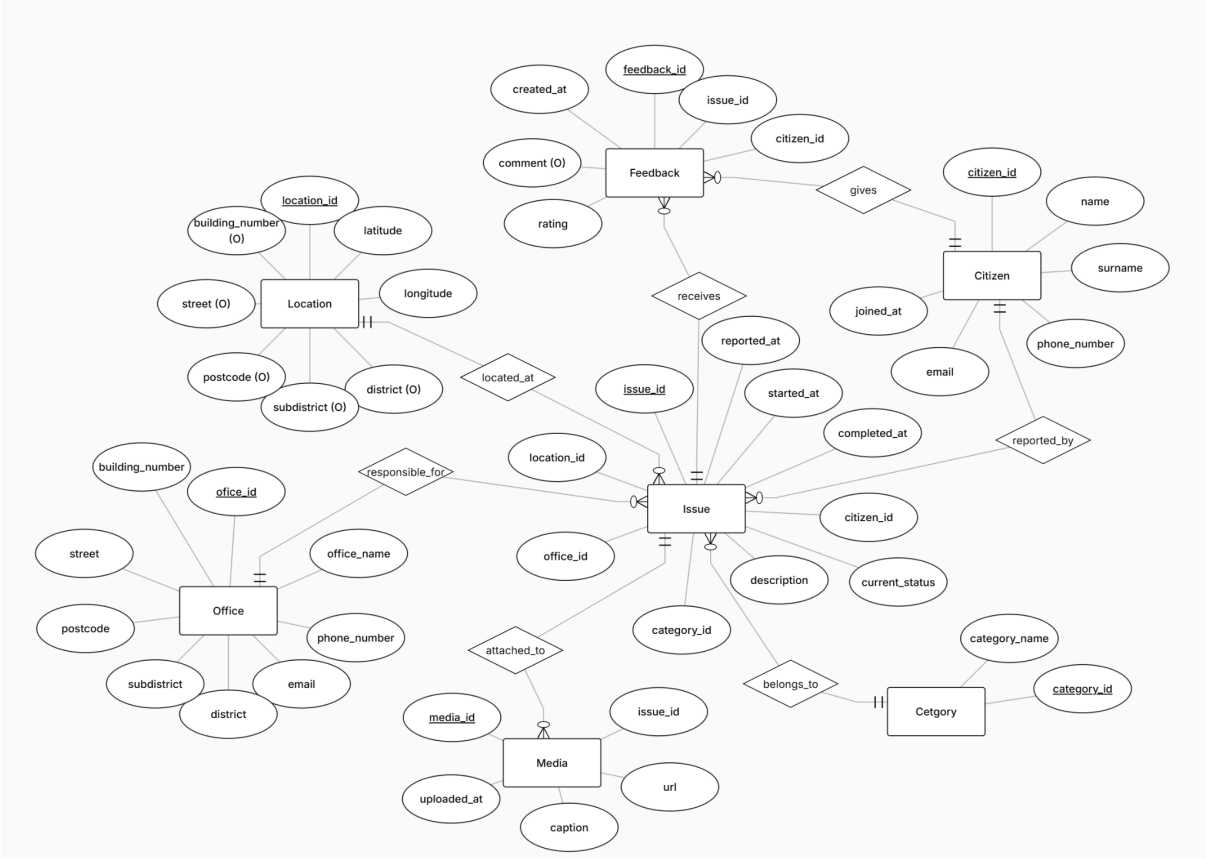
- **Feedback**

This one tells about the opinion of citizens after the issue was solved or want to comment anything. The data includes feedback ID, issue ID (link issue ID from issue table), citizen ID (link citizen ID form citizen table), comment, rating (1-5), and created at (date of comment).

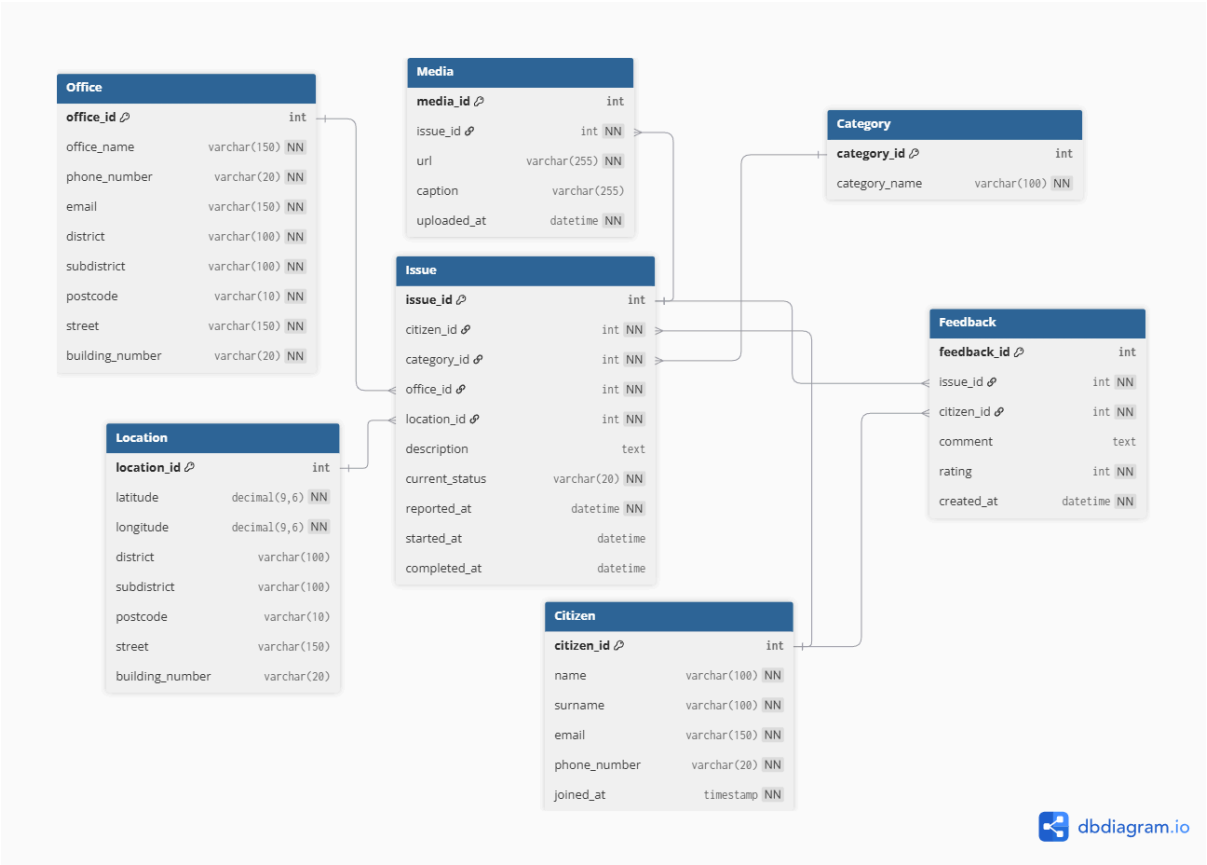
- **Media**

Media is the social media where people post comments. The topics include media ID, issue ID (link issue ID from issue table), url, caption, and uploaded at (date).

2.Conceptual design



3. Relational database schema diagram



The attribute current_status is shown as varchar(20) with allowed values in the diagram. In the actual MySQL implementation is defined as an NUM('pending', 'in_progress', 'resolved') with a default value of 'pending'. Dbddiagram.io does not support enum type.

4. Data dictionary

This data dictionary is the outlined structure of a citizen reporting system, linking citizens, issues, feedback, locations, media, categories, and offices and for knowing relationships between tables ensure traceability (e.g., who reported what, where, and how it was resolved).

1. Citizen (This table stores information about citizens who can report issues and provide feedback. **Purpose:** This table serves as the main reference for identifying who reports an issue or provides feedback)

Attribute	Data Type	Key	Description
citizen_id	INT	PK (Each citizen must have a **unique identifier** so that the system can distinguish one citizen from another, even if two citizens have the same name or email)	Unique identifier for each citizen
name	VARCHAR(100)		First name of the citizen
surname	VARCHAR(100)		Last name of the citizen
email	VARCHAR(150)	Unique	Citizen's email address
phone_number	VARCHAR(20)		Contact number of the citizen
joined_at	DATETIME		The date and time when the citizen was first registered or added to the system. It helps track when a citizen officially joined or became part of the database.

2. Issue (This table records all issues reported by citizens. **Purpose:** This table tracks issues end-to-end, linking the citizen, office, location, and category.)

Attribute	Data Type	Key	Description
issue_id	INT	PK (Every issue reported must have a **unique identifier** so the system can track it individually.)	Unique identifier for each reported issue
citizen_id	INT	FJ → Citizen.citizen_id	Citizen, who reported an issue

Attribute	Data Type	Key	Description
category_id	INT	FK → Category.category_id	Type/category of issue
office_id	INT	FK → Office.office_id	Office responsible
location_id	INT	FK → Location.location_id	Location of the issue
description	TEXT		Details of the issue
current_status	ENUM('pending', 'in_progress', 'resolved')		Status of issue completion
reported_at	DATETIME		Date and time issue was reported
started_at	DATATIME		This column stores the date and time when a specific process, task, or report was initiated. It helps track when the activity officially began in the system
completed_at	DATETIME		This column stores the date and time when a specific process, task, or report related to the citizen was finished. It helps track when the action was officially completed in the system

3. Feedback (This table collects feedback from citizens about reported issues. **Purpose:** Helps evaluate citizen satisfaction and the effectiveness of issue resolution)

Attribute	Data Type	Key	Description
feedback_id	INT	PK (Each feedback entry needs a **unique identifier**, because multiple citizens can give feedback on the same issue)	Unique feedback ID
issue_id	INT	FK → Issue.issue_id	Related issue
citizen_id	INT	FK → Citizen.citizen_id	Citizen giving feedback
comment	TEXT		Written feedback

Attribute	Data Type	Key	Description
rating	INT (1–5)		Rating given
created_at	DATETIME		Feedback submission date

4. Location (Stores geographic and address information. **Purpose:** Provides precise location data for reported issues.)

Attribut	Data Type	Key	Description
location_id	INT	PK (Each location must be uniquely identified even if two locations share similar street names or postal codes)	Unique location ID
latitude	DECIMAL(9,6)		Latitude coordinate
longitude	DECIMAL(9,6)		Longitude coordinate
district	VARCHAR(100)		District name
subdistrict	VARCHAR(100)		Subdistrict name
postcode	VARCHAR(10)		Postal code
street	VARCHAR(150)		Street name
building_number	VARCHAR(20)		Building/house number

5. Media (Stores multimedia files (images/videos) attached to issues. **Purpose:** Allows visual documentation of issues for better understanding and verification)

Attribute	Data Type	Key	Description
media_id	INT	PK(Every media file uploaded (image/video) needs a **unique identifier**)	Unique media ID
issue_id	INT	FK → Issue.issue_id	Related issue
url	VARCHAR(255)		Media file link (image/video)
caption	VARCHAR(255)		Media caption/description
uploaded_at	DATETIME		Upload timestamp

6. Category (Stores types or categories of issues. **Purpose:** Helps classify issues for easier reporting and management)

Attribute	Data Type	Key	Description
category_id	INT	PK (Each issue category (e.g., waste, flood, road damage) must have a **unique identifier**)	Unique category ID
category_name	VARCHAR(100)		Name of category (e.g. waste, road damage, flood)

7. Office (Stores information about offices responsible for handling reported issues. **Purpose:** Provides contact and location information for the responsible office for efficient issue resolution)

Attribute	Data Type	Key	Description
office_id	INT	PK (Each office must be uniquely identifiable, even if two offices have similar names or are in the same district)	Unique office ID
office_name	VARCHAR(150)		Name of responsible office
phone_number	VARCHAR(20)		Contact number
e_mail	VARCHAR(150)		Office email
district	VARCHAR(100)		District of the office
subdistrict	VARCHAR(100)		Subdistrict of the office
postcode	VARCHAR(10)		Postal code
street	VARCHAR(150)		Street name
building_number	VARCHAR(20)		Building/house number

SQL DDL AND SAMPLE DATA

```
CREATE DATABASE project_1
CHARACTER SET utf8mb4
COLLATE utf8mb4_general_ci;

USE project_1;

SET NAMES utf8mb4;
```

```
CREATE TABLE `Citizen` (
  citizen_id    INT PRIMARY KEY AUTO_INCREMENT,
  name          VARCHAR(100)       NOT NULL,
  surname       VARCHAR(100)       NOT NULL,
  email         VARCHAR(150)       NOT NULL UNIQUE,
  phone_number  VARCHAR(20) NOT NULL,
  joined_at     DATETIME           NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

insert into Citizen (name, surname, email, phone_number, joined_at) values
('Maurise', 'Ianniello', 'mianniello0@umich.edu', '120-289-9353', '2022-01-09'),
('Josephine', 'Fosbraey', 'jfosbraey1@hexun.com', '696-627-0797', '2022-01-06'),
('Meier', 'Tottie', 'mtottie2@plala.or.jp', '618-346-7216', '2021-03-15'),
('Grange', 'Ned', 'nglasard0@opera.com', '206-230-6409', '2024-12-07'),
('Kellen', 'Flemming', 'fheyball1@yale.edu', '964-718-0271', '2020-04-28'),
('Zacherie', 'Parsifal', 'pvanderbrug3@opera.com', '848-273-2669', '2024-03-23'),
('Dedra', 'Sterling', 'shacksby4@dagondesign.com', '753-237-3935', '2025-02-13'),
('Mikol', 'Wainwright', 'wbaddiley5@pen.io', '944-559-3634', '2024-03-30'),
('Deirdre', 'Starla', 'spennini6@rakuten.co.jp', '389-282-0490', '2022-07-11'),
('Tomaso', 'Wolfy', 'wcripwell7@gov.uk', '579-325-7096', '2025-04-21'),
('Melody', 'Paulie', 'pnaton8@soundcloud.com', '774-845-9816', '2024-09-20'),
('Gladi', 'Karlen', 'kaiersa@mtv.com', '450-142-1532', '2022-01-14'),
('Rakel', 'Welsh', 'wchesona@nba.com', '669-440-4725', '2024-08-18'),
('Camilla', 'Tedd', 'twoolagerd@nasa.gov', '549-488-3864', '2022-03-11'),
('Ailis', 'Ab', 'amartonf@networkadvertising.org', '494-973-4470', '2020-12-16'),
('Egon', 'Richmond', 'rbluemang@soundcloud.com', '680-852-0161', '2020-01-14'),
('Riva', 'Alison', 'adurkinh@google.co.uk', '800-392-0756', '2021-01-27'),
('Roxane', 'Constantina', 'cbettleyi@slideshare.net', '561-993-5137', '2024-08-12'),
('Sarette', 'Kimmi', 'ksorleyj@adobe.com', '632-406-8014', '2020-12-22'),
('Ileana', 'Bradan', 'bbaguleyk@dion.ne.jp', '289-760-7376', '2024-09-30'),
('Sephira', 'Jobie', 'jchansonnaul@seattletimes.com', '588-330-0474', '2020-08-31'),
```

```
('Jessy', 'Flinn', 'fgentzschm@posterous.com', '641-524-7309', '2020-03-22'),
('Derrick', 'Timothea', 'tbuckneyn@foxnews.com', '281-231-8495', '2023-02-16'),
('Dulcie', 'Jehu', 'jparisho@biblegateway.com', '718-149-1115', '2025-07-15'),
('Devland', 'Carissa', 'ckobap@loc.gov', '339-596-2200', '2023-11-12'),
('Calvin', 'Edgar', 'eschimannq@google.fr', '920-513-7097', '2021-04-17'),
('Lynnelle', 'Terrie', 'tdraycottr@walmart.com', '889-904-8038', '2024-12-25'),
('Aylmer', 'Raoul', 'rlongstaffes@taobao.com', '799-313-9252', '2020-11-24'),
('Othilia', 'Emalee', 'eadshedet@qq.com', '271-596-6133', '2022-02-28'),
('Keven', 'Gene', 'gguerrazzi10@yandex.ru', '493-206-0689', '2020-09-07'),
('Nannie', 'Chlo', 'cgreated11@hostgator.com', '843-285-5996', '2023-08-30'),
('Brooks', 'Spencer', 'sjacqueme12@answers.com', '675-454-6810', '2023-04-01'),
('Ophelie', 'Malanie', 'myurtsev13@ask.com', '962-862-8268', '2022-10-15'),
('Fanchon', 'Elaine', 'ebecke14@oakley.com', '893-767-9118', '2024-05-27'),
('Alec', 'Rickie', 'rstokey15@usatoday.com', '767-845-1868', '2021-04-08'),
('Mollee', 'Vania', 'vspofford16@indiegogo.com', '336-419-4419', '2021-05-22');

CREATE TABLE `Category` (
  category_id    INT PRIMARY KEY AUTO_INCREMENT,
  category_name  VARCHAR(100)      NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

insert into Category (category_name) values
        ('Transportations'),
        ('Utilities'),
         ('Waste'),
        ('Crimes'),
        ('Health'),
        ('Accident'),
        ('Environment'),
        ('Disaster');

CREATE TABLE Office (
```

```
  office_id       INT PRIMARY KEY AUTO_INCREMENT,
  office_name     VARCHAR(150)    NOT NULL,
  phone_number    VARCHAR(20)   NOT NULL,
  e_mail          VARCHAR(150)      NOT NULL,
  district        VARCHAR(100)      NOT NULL,
  subdistrict     VARCHAR(100)    NOT NULL,
  postalcode      VARCHAR(10)     NOT NULL,
  street          VARCHAR(150)      NOT NULL,
  building_number  VARCHAR(20)           NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

insert into Office (office_name, phone_number, e_mail, district, subdistrict, postalcode, street,
building_number) values
('Transportations Department', '111-111-1111', 'cmatthis7@wsj.com', 'Khlong Luang District', 'Khlong
Nueng', '12120', 'Phahonyothin Rd', 10),
('Health Department', '111-111-1112', 'csatthis7@wsj.com', 'Khlong Luang District', 'Khlong Nueng',
'12120', 'Phahonyothin Rd', 12),
('Utilities Department', '111-111-113', 'mkevlin1@themeforest.net', 'Khlong Luang District', 'Khlong
Nueng', '12120', 'Phahonyothin Rd', 3),
('Waste Department', '111-111-114', 'ctinson0@wix.com', 'Khlong Luang District', 'Khlong Nueng',
'12120', 'Phahonyothin Rd', 4),
('Crimes Department', '111-111-115', 'bchestnut2@gravatar.com', 'Khlong Luang District', 'Khlong
Nueng', '12120', 'Phahonyothin Rd', 5),
('Accident Department', '111-111-116', 'hmunro3@webeden.co.uk', 'Khlong Luang District', 'Khlong
Nueng', '12120', 'Phahonyothin Rd', 6),
('Environment Department', '111-111-117', 'kderges4@taobao.com', 'Khlong Luang District', 'Khlong
Nueng', '12120', 'Phahonyothin Rd', 7),
('Disaster Department', '111-111-118', 'bmaynard5@discovery.com', 'Khlong Luang District', 'Khlong
Nueng', '12120', 'Phahonyothin Rd', 8);

CREATE TABLE Location (
  location_id     INT PRIMARY KEY AUTO_INCREMENT,
  latitude        DECIMAL(9,6)        NOT NULL,
  longitude       DECIMAL(10,7)     NOT NULL,
  district        VARCHAR(100),
```

```
  subdistrict     VARCHAR(100),
  postcode        VARCHAR(10),
  street          VARCHAR(150),
  building_number VARCHAR(20),
  CONSTRAINT ck_location_lat CHECK (latitude  IS NULL OR (latitude BETWEEN -90  AND 90)),
  CONSTRAINT ck_location_lon CHECK (longitude IS NULL OR (longitude BETWEEN -180 AND 180))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

INSERT INTO Location (latitude, longitude, district, subdistrict, postcode, street, building_number)
VALUES
(14.563054, 121.031497, 'Khlong Luang District', 'Khlong Nueng', '12120', 'Phahonyothin Road', '10'),
(-43.260364, -65.358137, 'Khlong Luang District', 'Khlong Nueng', '12120', 'Phahonyothin Road', '12'),
(39.2699174, -9.1052924, 'Lam Luk Ka', 'Bueng Kham Phroi', '12150', 'Lam Luk Ka Road', '538'),
(-9.770221, 124.514063, 'Lat Lum Kaeo', 'Khu Bang Luang', '12140', 'Lam Luk Ka Road', '76'),
(27.730381, 112.006998, 'Khlong Luang', 'Khlong Sam', '12120', 'Wat Suk Chai Road', '510'),
(-0.532915, 166.944141, 'Mueang Pathum Thani', 'Bang Duea', '12000', 'Frontage Road', '619'),
(43.575754, 21.331050, 'Lat Lum Kaeo', 'Rahaeng', '12140', 'Pathum Thani-Bang Len Road', '721'),
(-7.528417, 107.734013, 'Nong Suea', 'Bueng Ba', '12170', 'Ratchaphruek Road', '63'),
(38.889471, -77.057855, 'Nong Suea', 'Noppharat', '12170', 'Vibhavadi Rangsit Road', '740'),
(30.542510, 119.977400, 'Thanyaburi', 'Rangsit', '12130', 'Motorway No. 5', NULL),
(15.165090, 120.657329, 'Sam Khok', 'Sam Khok', '12160', 'Phahon Yothin Road', '412'),
(50.499760, -120.802530, 'Nong Suea', 'Nong Sam Wang', '12170', 'Nimitmai Road', '903'),
(39.672589, 66.772463, 'Thanyaburi', 'Rangsit', '12130', 'Rangsit-Nakhon Nayok Road', '285'),
(49.615510, 101.992797, 'Mueang Pathum Thani', 'Bang Khayaeng', '12000', 'Highway No. 3214', NULL),
(9.137597, -75.212896, 'Lat Lum Kaeo', 'Khu Bang Luang', '12000', 'Kanchanaphisek Road', '552'),
(59.276517, 15.207783, 'Khlong Luang', 'Khlong Si', '12120', 'Nimit Mai Road', '870'),
(13.512832, 122.469532, 'Lam Luk Ka', 'Lat Sawai', '12150', 'Lam Luk Ka Road', '29'),
(44.638131, 41.938156, 'Mueang Pathum Thani', 'Bang Khayaeng', '12000', 'Vibhavadi Rangsit Road', '871'),
(12.534030, 5.783050, 'Mueang Pathum Thani', 'Bang Kadi', '12000', 'Vibhavadi Rangsit Road', '729'),
(-7.461263, 109.220798, 'Nong Suea', 'Sala Khru', '12170', 'Hathai Rat Road', '11'),
(-2.715840, 30.628610, 'Nong Suea', 'Bueng Bon', '12170', 'Rangsit-Nakhon Nayok Road', '215'),
```

```
(19.739176, -71.444737, 'Thanyaburi', 'Prachathipat', '12130', 'Pridi Banomyong Road', '362'),
(-20.674197, -40.499738, 'Thanyaburi', 'Bueng Nam Rak', '12110', 'Highway No. 346', NULL),
(6.605868, -0.711713, 'Sam Khok', 'Bang Toei', '12160', 'Nimit Mai Road', '92'),
(35.293048, 112.390857, 'Nong Suea', 'Noppharat', '12170', 'Ratchaphruek Road', '124'),
(0.207025, 32.540102, 'Khlong Luang', 'Khlong Sam', '12120', 'Nimit Mai Road', '934'),
(41.933917, 126.422806, 'Lam Luk Ka', 'Bueng Thong Lang', '12150', 'Lam Luk Ka Road', '83'),
(24.880990, 118.718990, 'Khlong Luang', 'Khlong Song', '12120', 'Vibhavadi Rangsit Road', '843'),
(14.559360, 121.041239, 'Thanyaburi', 'Lam Phak Kut', '12110', 'Pridi Banomyong Road', '672'),
(35.621960, 118.725698, 'Nong Suea', 'Bueng Ka Sam', '12170', 'Kanchanaphisek Road', '82'),
(30.456360, 119.117600, 'Mueang Pathum Thani', 'Lak Hok', '12130', 'Ratchaphruek Road', '743'),
(29.833390, 108.731120, 'Mueang Pathum Thani', 'Ban Mai', '12000', 'Pridi Banomyong Road', '377'),
(49.858670, 18.977085, 'Khlong Luang', 'Khlong Ha', '12120', 'Rangsit-Nakhon Nayok Road', '031'),
(24.410250, 117.416058, 'Lam Luk Ka', 'Phuet Udom', '12150', 'Lam Luk Ka Road', '903'),
(26.141850, 116.188400, 'Mueang Pathum Thani', 'Bang Kadi', '12000', 'Hathai Rat Road', '431'),
(36.834080, 10.040570, 'Lat Lum Kaeo', 'Na Mai', '12140', 'Lam Luk Ka Road', '830'),
(27.275370, 91.239777, 'Sam Khok', 'Chiang Rak Yai', '12160', 'Kanchanaphisek Road', '341');

CREATE TABLE `Issue` (
  issue_id        INT PRIMARY KEY AUTO_INCREMENT,
  citizen_id      INT             NOT NULL,
  category_id     INT             NOT NULL,
  office_id       INT             NOT NULL,
  location_id     INT             NOT NULL,
  description     TEXT,
  current_status  ENUM('pending', 'in_progress', 'resolved') NOT NULL DEFAULT 'pending',
  reported_at     DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
  started_at      DATETIME NULL,
  completed_at    DATETIME NULL,

  CONSTRAINT fk_issue_citizen   FOREIGN KEY (citizen_id)  REFERENCES Citizen(citizen_id),
  CONSTRAINT fk_issue_category  FOREIGN KEY (category_id) REFERENCES Category(category_id),
  CONSTRAINT fk_issue_office    FOREIGN KEY (office_id)   REFERENCES Office(office_id),
```

```
  CONSTRAINT fk_issue_location  FOREIGN KEY (location_id) REFERENCES Location(location_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;




DELIMITER //

CREATE TRIGGER trg_issue_completed_at
BEFORE UPDATE ON Issue
FOR EACH ROW
BEGIN
  IF NEW.current_status = 'resolved' AND (NEW.completed_at IS NULL) THEN
    SET NEW.completed_at = NOW();
    END IF;
END//

DELIMITER ;

DELIMITER //

CREATE TRIGGER trg_issue_started_at
BEFORE UPDATE ON Issue
FOR EACH ROW
BEGIN
  IF NEW.current_status = 'in_progress' AND (OLD.current_status = 'pending') AND NEW.started_at IS
NULL THEN
    SET NEW.started_at = NOW();
  END IF;
END//

DELIMITER ;
```

INSERT INTO Issue (category_id, office_id, location_id, citizen_id, description, current_status, reported_at, started_at, completed_at) VALUES

(7, 1, 5, 1, 'Afewtu', 'resolved', '2023-10-25', '2023-12-07', '2024-01-24'),

(1, 5, 36, 2, 'Wzgdje', 'in_progress', '2022-11-05', '2022-11-06', NULL),

(8, 7, 23, 3, 'Pcsxpm', 'resolved', '2023-12-14', '2023-12-25', '2024-03-13'),

(2, 6, 3, 4, 'Tjlbji', 'pending', '2020-04-07', '2020-06-06', '2020-09-01'),

(1, 2, 31, 5, 'Aewgft', 'resolved', '2021-06-10', '2021-08-17', '2021-09-03'),

(7, 5, 14, 6, 'Duqsyw', 'pending', '2025-06-03', NULL, NULL),

(7, 4, 28, 7, 'Nhpsnw', 'in_progress', '2023-09-13', '2023-09-23', NULL),

(8, 3, 32, 8, 'Daycug', 'resolved', '2024-12-23', '2024-12-31', '2025-04-25'),

(8, 2, 3, 9, 'Mdipfp', 'resolved', '2021-04-22', '2021-06-12', '2021-08-27'),

(8, 2, 20, 10,'Ivwwuo', 'resolved', '2022-10-03', '2022-12-11', '2023-02-07'),

(4, 5, 3, 11,'Nextgd', 'resolved', '2021-01-08', '2021-03-09', '2021-03-24'),

(8, 7, 23, 12,'Vimdkc', 'resolved', '2021-03-13', '2021-04-23', '2021-06-21'),

(8, 4, 11, 13,'Yhufjy', 'resolved', '2023-06-04', '2023-06-29', '2023-10-05'),

(6, 7, 23, 14,'Hrnrhk', 'resolved', '2022-05-14', '2022-06-18', '2022-10-09'),

(1, 8, 24, 15,'Wcdjqz', 'resolved', '2023-02-21', '2023-04-01', '2023-05-17'),

(8, 3, 2, 16,'Eaonke', 'resolved', '2024-07-20', '2024-09-19', '2024-11-11'),

(3, 3, 13, 17,'Ciakaz', 'in_progress', '2025-01-02', '2025-01-21', '2025-04-23'),

(8, 7, 29, 18,'Mlunsl', 'in_progress', '2023-02-27', '2023-04-14', NULL),

(1, 8, 2, 19,'Ewsbnb', 'resolved', '2023-12-12', NULL, NULL),

(6, 4, 28, 20,'Xrrpvd', 'in_progress', '2022-10-27', '2022-11-20', NULL),

(2, 1, 1, 21,'Gzdqif', 'pending', '2024-03-08', '2024-03-10', '2024-07-11'),

(7, 1, 34, 22,'Vbqsau', 'resolved', '2021-05-24', '2021-06-23', '2021-08-29'),

(6, 6, 29, 23,'Ypddbg', 'in_progress', '2022-03-14', '2022-05-06', NULL),

(2, 8, 30, 24,'Oqbjbk', 'resolved', '2021-08-03', '2021-08-23', '2021-11-17'),

(7, 8, 20, 25,'Srarvr', 'pending', '2020-10-21', NULL, NULL),

(1, 6, 37, 26,'Cbedxg', 'resolved', '2021-03-28', '2021-05-03', '2021-06-19'),

(5, 2, 24, 27,'Ljifkh', 'resolved', '2020-07-16', '2020-07-30', '2020-11-17'),

(3, 5, 19, 28,'Iolfmz', 'in_progress', '2020-09-19', '2020-10-21', '2021-01-23'),

(5, 2, 10, 29,'Zhnbjg', 'pending', '2023-01-26', '2023-03-14', '2023-06-21'),

```
(4, 3, 26, 30,'Vbjcal', 'pending',     '2023-12-28', NULL,          NULL),
(5, 1, 34, 31,'Orauzf', 'in_progress', '2024-08-23', '2024-11-02', NULL),
(1, 3, 14, 32,'Zugnre', 'pending',     '2025-07-29', '2025-09-30', '2025-11-23'),
(4, 4, 18, 33,'Gbafpb', 'in_progress', '2021-09-11', '2021-09-22', '2022-01-30'),
(4, 6, 34, 34,'Ojqnun', 'resolved',    '2022-07-20', '2022-08-03', '2022-11-19'),
(1, 2, 9,  35,'Lieqkp', 'pending',     '2022-06-17', NULL,          NULL),
(3, 6, 7,  36,'Rpyzbq', 'pending',     '2020-01-01', NULL,          NULL),
(6, 7, 26,  1,'Teclsj', 'resolved',    '2022-06-03', '2022-08-14', '2022-08-31'),
(7, 2, 37,  2,'Mljdgi', 'resolved',    '2024-11-16', '2024-12-18', '2025-03-20'),
(4, 4, 7,   3,'Vppdkx', 'in_progress', '2021-12-29', '2022-02-06', NULL),
(1, 1, 30,  4,'Wspolr', 'resolved',    '2022-08-09', '2022-09-11', '2022-12-24'),
(1, 4, 4,   5,'Ahtfwj', 'resolved',    '2023-11-13', '2023-12-19', '2024-04-03'),
(3, 1, 10,  6,'Qktebd', 'resolved',    '2022-10-07', '2022-11-28', '2023-01-04'),
(2, 5, 22,  7,'Lojmbc', 'resolved',    '2023-03-19', '2023-05-28', '2023-08-02'),
(1, 3, 11,  8,'Wqnnnv', 'in_progress', '2021-11-06', '2021-11-28', NULL),
(5, 5, 2,   9,'Elqcvc', 'resolved',    '2020-10-07', '2020-10-24', '2021-01-23'),
(6, 8, 37, 10,'Gtfjwy', 'resolved',    '2022-11-22', '2022-12-14', '2023-04-09'),
(7, 2, 20, 11,'Njselm', 'resolved',    '2020-03-16', '2020-05-19', '2020-07-04'),
(7, 8, 5,  12,'Rlxuza', 'resolved',    '2021-07-12', '2021-08-12', '2021-11-20'),
(8, 8, 32, 13,'Czkbqr', 'resolved',    '2022-04-10', '2022-05-05', '2022-08-09'),
(1, 7, 13, 14,'Buedjf', 'resolved',    '2020-10-02', '2020-11-05', '2021-02-13');


CREATE TABLE Feedback (
  feedback_id  INT PRIMARY KEY AUTO_INCREMENT,
  issue_id     INT              NOT NULL,
  citizen_id   INT              NOT NULL,
  comment      TEXT,
  rating       INT              NOT NULL,
  created_at   DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
CONSTRAINT fk_feedback_issue   FOREIGN KEY (issue_id)   REFERENCES Issue(issue_id)   ON DELETE CASCADE,

CONSTRAINT fk_feedback_citizen FOREIGN KEY (citizen_id) REFERENCES Citizen(citizen_id) ON DELETE CASCADE,

CONSTRAINT ck_feedback_rating  CHECK (rating BETWEEN 1 AND 5)

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;


INSERT INTO Feedback (feedback_id, issue_id, citizen_id, comment, rating, created_at) VALUES

(1, 4, 13, 'The road has been under construction for weeks, no visible progress.', 1, '2025-08-23'),

(2, 28, 33, 'Very quick response from the office, thank you!', 5, '2025-03-26'),

(3, 39, 21, 'Streetlights fixed within two days, great job.', 4, '2024-09-02'),

(4, 13, 28, 'The problem was solved but took longer than expected.', 4, '2025-04-17'),

(5, 44, 29, 'Garbage collection service is inconsistent, sometimes delayed.', 3, '2025-07-10'),

(6, 34, 29, 'Flooding issue addressed properly, good communication.', 5, '2024-10-24'),

(7, 32, 13, 'Still waiting for action, nothing has changed so far.', 1, '2025-03-10'),

(8, 45, 11, 'Hospital staff were kind and helpful during the emergency.', 3, '2025-02-05'),

(9, 16, 2, 'Broken traffic lights repaired quickly, very satisfied.', 5, '2024-11-29'),

(10, 16, 20, 'Took too long to resolve, unsafe for pedestrians.', 2, '2024-10-27'),

(11, 22, 30, 'The issue was fixed but I had to call several times.', 3, '2025-01-13'),

(12, 28, 33, 'Quick response and clear updates provided.', 1, '2024-12-27'),

(13, 28, 12, 'Clean-up handled well, no problems anymore.', 5, '2024-10-04'),

(14, 38, 8, 'Waste bins are still overflowing, not resolved.', 1, '2025-02-25'),

(15, 9, 35, 'The police arrived late, but they solved the situation.', 3, '2025-01-29'),

(16, 41, 34, 'Service was acceptable, but could be faster.', 3, '2025-01-20'),

(17, 17, 32, 'Appreciate the updates, though resolution was slow.', 2, '2025-03-28'),

(18, 44, 27, 'Good coordination between offices, well done.', 1, '2025-01-24'),

(19, 47, 12, 'Problem solved on the same day, excellent work!', 5, '2025-06-09'),

(20, 44, 25, 'Communication was poor, had to follow up multiple times.', 4, '2025-05-17');
```

```
CREATE TABLE `Media` (
  media_id     INT PRIMARY KEY AUTO_INCREMENT,
  issue_id     INT              NOT NULL,
  url          VARCHAR(255)     NOT NULL,
  caption      VARCHAR(255),
  uploaded_at  DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP,

CONSTRAINT fk_media_issue FOREIGN KEY (issue_id) REFERENCES Issue(issue_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

INSERT INTO Media (issue_id, url, caption, uploaded_at) VALUES
(1,  'https://imgs.example.com/transport/road-001.jpg',     'Pothole near school entrance #Transportations', '2024-01-15'),
(2,  'https://imgs.example.com/utilities/power-001.jpg',    'Fallen electric pole #Utilities', '2024-03-03'),
(3,  'https://imgs.example.com/waste/bin-001.jpg',          'Overflowing garbage bins #Waste', '2024-06-18'),
(4,  'https://imgs.example.com/crime/graffiti-001.jpg',     'Graffiti on community hall #Crimes', '2024-08-27'),
(5,  'https://imgs.example.com/health/clinic-001.jpg',      'Broken wheelchair ramp #Health', '2024-09-12'),
(6,  'https://imgs.example.com/accident/sign-001.jpg',      'Damaged stop sign #Accident', '2024-10-30'),
(7,  'https://imgs.example.com/env/tree-001.jpg',           'Fallen tree blocking road #Environment', '2024-11-19'),
(8,  'https://imgs.example.com/disaster/flood-001.jpg',     'Street flooded after storm #Disaster', '2024-12-05'),
(9,  'https://imgs.example.com/transport/busstop-001.jpg',  'Broken bench at bus stop #Transportations',    '2025-01-14'),
(10, 'https://imgs.example.com/utilities/water-001.jpg',    'Water pipe leakage #Utilities', '2025-02-02'),
(11, 'https://imgs.example.com/waste/dump-001.jpg',         'Illegal dumping in alley #Waste', '2025-02-25'),
(12, 'https://imgs.example.com/crime/light-001.jpg',        'Streetlight not working #Crimes', '2025-03-08'),
```

(13, 'https://imgs.example.com/health/park-001.jpg', 'Dirty public restroom #Health', '2025-03-21'),

(14, 'https://imgs.example.com/accident/crosswalk-001.jpg', 'Faded crosswalk lines #Accident', '2025-04-02'),

(15, 'https://imgs.example.com/env/smoke-001.jpg', 'Smoke from burning waste #Environment', '2025-04-15'),

(16, 'https://imgs.example.com/disaster/wind-001.jpg', 'Roof damaged by strong winds #Disaster', '2025-05-06'),

(17, 'https://imgs.example.com/transport/bridge-001.jpg', 'Cracks in pedestrian bridge #Transportations', '2025-05-20'),

(18, 'https://imgs.example.com/utilities/gas-001.jpg', 'Gas leak suspected #Utilities', '2025-06-01'),

(19, 'https://imgs.example.com/waste/container-001.jpg', 'Recycling container missing #Waste', '2025-06-18'),

(20, 'https://imgs.example.com/crime/vandalism-001.jpg', 'Vandalized playground #Crimes', '2025-07-02');

SQL commands

1. **The citizen report a new issue to government of Pathumthani:**

INSERT Issue (citizen_id, category_id, office_id, location_id, description, current_status,

reported_at, started_at, completed_at) values

(1 ,3, 4, 32, 'feowejwiew', 'pending', '2025-08-03', NULL, NULL);

```
USE `project_1`;
INSERT Issue (citizen_id, category_id, office_id, location_id, description, current_status,
reported_at, started_at, completed_at) values
(1 ,3, 4, 32, 'feowejwiew', 'pending', '2025-08-03', NULL, NULL);
```

Output

Action Output

#	Time	Action	Message
1	13:05:34	USE `project_1`	0 row(s) affected
2	13:05:34	INSERT Issue (citizen_id, category_id, office_id, location_id, description, current_statu...	1 row(s) affected

2. **Show the total of issues by categories for each district:**

SELECT

l.district,

SUM(CASE WHEN i.category_id = 1 THEN 1 ELSE 0 END) AS Transportation,

SUM(CASE WHEN i.category_id = 2 THEN 1 ELSE 0 END) AS Utilities,

SUM(CASE WHEN i.category_id = 3 THEN 1 ELSE 0 END) AS Waste,

SUM(CASE WHEN i.category_id = 4 THEN 1 ELSE 0 END) AS Crimes,

SUM(CASE WHEN i.category_id = 5 THEN 1 ELSE 0 END) AS Health,

SUM(CASE WHEN i.category_id = 6 THEN 1 ELSE 0 END) AS Accident,

SUM(CASE WHEN i.category_id = 7 THEN 1 ELSE 0 END) AS Environment,

SUM(CASE WHEN i.category_id = 8 THEN 1 ELSE 0 END) AS Disaster

FROM Issue i

JOIN Location l on l.location_id = i.location_id

JOIN Category c on c.category_id = i.category_id

GROUP BY l.district

ORDER BY l.district;

```sql
SELECT
    l.district,
    SUM(CASE WHEN i.category_id = 1 THEN 1 ELSE 0 END) AS Transportation,
    SUM(CASE WHEN i.category_id = 2 THEN 1 ELSE 0 END) AS Utilities,
    SUM(CASE WHEN i.category_id = 3 THEN 1 ELSE 0 END) AS Waste,
    SUM(CASE WHEN i.category_id = 4 THEN 1 ELSE 0 END) AS Crimes,
    SUM(CASE WHEN i.category_id = 5 THEN 1 ELSE 0 END) AS Health,
    SUM(CASE WHEN i.category_id = 6 THEN 1 ELSE 0 END) AS Accident,
    SUM(CASE WHEN i.category_id = 7 THEN 1 ELSE 0 END) AS Environment,
    SUM(CASE WHEN i.category_id = 8 THEN 1 ELSE 0 END) AS Disaster
FROM Issue i
JOIN Location l on l.location_id = i.location_id
JOIN Category c on c.category_id = i.category_id
GROUP BY l.district
ORDER BY l.district;
```

district	Transportation	Utilities	Waste	Crimes	Health	Accident	Environment	Disaster
Khlong Luang	0	0	0	1	0	2	3	0
Khlong Luang District	1	1	0	0	1	0	0	1
Lam Luk Ka	0	1	0	2	1	0	1	1
Lat Lum Kaeo	2	0	1	1	0	0	0	0
Mueang Pathum Thani	2	0	1	1	0	0	1	2
Nong Suea	2	1	0	0	0	0	2	1
Sam Khok	3	0	0	0	1	1	1	1
Thanyaburi	1	1	2	0	1	2	0	3

3. Number of ratings for each rating score:

SELECT rating, COUNT(rating) AS total_rating

FROM Feedback

GROUP BY rating

ORDER BY rating ASC;

```sql
SELECT rating, COUNT(rating) AS total_rating
FROM Feedback
GROUP BY  rating
ORDER BY rating ASC;
```

Result Grid | Filter Rows: | Export: | Wrap

rating	total_rating
1	5
2	2
3	5
4	3
5	5

4. **Top 5 longest unresolved issues:**

```
SELECT
   i.issue_id,
   c.category_name,
   l.district,
   i.current_status,
   DATEDIFF(CURDATE(), i.reported_at) AS days_open
FROM Issue i
JOIN Location l ON i.location_id = l.location_id
JOIN Category c ON c.category_id = i.category_id
WHERE i.current_status != 'resolved'
ORDER BY days_open DESC
LIMIT 5;
```

```
SELECT
    i.issue_id,
    c.category_name,
    l.district,
    i.current_status,
    DATEDIFF(CURDATE(), i.reported_at) AS days_open
FROM Issue i
JOIN Location l ON i.location_id = l.location_id
JOIN Category c ON c.category_id = i.category_id
WHERE i.current_status != 'resolved'
ORDER BY days_open DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch ro

issue_id	category_name	district	current_status	days_open
36	Waste	Lat Lum Kaeo	pending	2075
4	Utilities	Lam Luk Ka	pending	1978
28	Waste	Mueang Pathum Thani	in_progress	1813
25	Environment	Nong Suea	pending	1781
33	Crimes	Mueang Pathum Thani	in_progress	1456

5. **Count the number of issues which are completed and incompleted**

```
SELECT
    o.office_name,
    COUNT(CASE WHEN i.current_status = 'resolved' THEN 1 END) AS total_resolved,
    COUNT (CASE WHEN i.current_status = 'in_progress' THEN 1 END) + COUNT (CASE WHEN
    i.current_status = 'pending' THEN 1 END) AS total_unresolved
FROM Issue i
JOIN Office o ON i.office_id = o.office_id
GROUP BY o.office_name;
```

office_name	total_resolved	total_unresolved
Transportations Department	4	2
Health Department	6	2
Utilities Department	2	4
Waste Department	2	4
Crimes Department	3	3
Accident Department	2	3
Environment Department	5	1
Disaster Department	6	1

6. **Identifies categories where government struggles (many reports, low rating, long resolution time) → useful for policy prioritization.**

```
SELECT c.category_name,
    COUNT(i.issue_id) AS total_reports,
    ROUND(AVG(f.rating), 2) AS avg_rating,
    ROUND(AVG(TIMESTAMPDIFF(DAY, i.reported_at, i.completed_at)), 1) AS
avg_resolution_days
FROM Issue i
JOIN Category c ON i.category_id = c.category_id
LEFT JOIN Feedback f ON i.issue_id = f.issue_id
GROUP BY c.category_name
HAVING COUNT(i.issue_id) > 5
ORDER BY total_reports DESC, avg_rating ASC;
```

```sql
use project_1;
SELECT c.category_name,
       COUNT(i.issue_id) AS total_reports,
       ROUND(AVG(f.rating), 2) AS avg_rating,
       ROUND(AVG(TIMESTAMPDIFF(DAY, i.reported_at, i.completed_at)), 1) AS avg_resolution_days
FROM Issue i
JOIN Category c ON i.category_id = c.category_id
LEFT JOIN Feedback f ON i.issue_id = f.issue_id
GROUP BY c.category_name
HAVING COUNT(i.issue_id) > 5
ORDER BY total_reports DESC, avg_rating ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category_name	total_reports	avg_rating	avg_resolution_days
Transportations	13	2.40	111.9
Disaster	10	3.50	115.4
Environment	8	3.00	110.6
Waste	6	3.25	115.6

7. **Creates a composite performance index balancing speed & satisfaction, so officials see not just "resolved issues," but *quality of resolution***

```sql
SELECT o.office_name,
    COUNT(i.issue_id) AS total_resolved,
    ROUND(AVG(f.rating), 2) AS avg_rating,
    ROUND(AVG(TIMESTAMPDIFF(DAY, i.reported_at, i.completed_at)), 1) AS avg_resolution_days,
    (ROUND(AVG(f.rating), 2) * 100 / (1 + ROUND(AVG(TIMESTAMPDIFF(DAY, i.reported_at, i.completed_at)), 1))) AS performance_index
FROM Office o
JOIN Issue i ON o.office_id = i.office_id
LEFT JOIN Feedback f ON i.issue_id = f.issue_id
WHERE i.current_status = 'resolved'
GROUP BY o.office_name
ORDER BY performance_index DESC;
```

```
9     LEFT JOIN feedback f ON i.issue_id = f.issue_id
10    WHERE i.current_status = 'resolved'
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

office_name	total_resolved	avg_rating	avg_resolution_days	performance_index
Accident Department	2	5.00	102.5	4.830918
Utilities Department	3	3.50	117.0	2.966102
Transportations Department	4	3.00	103.5	2.870813
Crimes Department	3	3.00	106.3	2.795899
Waste Department	2	3.50	132.5	2.621723
Health Department	6	3.00	116.2	2.559727
Environment Department	5	NULL	112.2	NULL
Disaster Department	6	NULL	116.2	NULL

8. **Provides transparency — citizens see which services are fast and which are slow, Encourages fairness and efficiency.**

```
SELECT c.category_name,
    ROUND(AVG(TIMESTAMPDIFF(HOUR, i.reported_at, i.completed_at)), 1) AS avg_waiting_hours,
    MAX(TIMESTAMPDIFF(HOUR, i.reported_at, i.completed_at)) AS longest_wait,
    MIN(TIMESTAMPDIFF(HOUR, i.reported_at, i.completed_at)) AS shortest_wait
FROM Issue i
JOIN Category c ON i.category_id = c.category_id
WHERE i.completed_at IS NOT NULL
GROUP BY c.category_name
ORDER BY avg_waiting_hours DESC;
```

```
10
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category_name	avg_waiting_hours	longest_wait	shortest_wait
Utilities	3084.0	3528	2544
Health	3024.0	3504	2592
Accident	3000.0	3552	2136
Disaster	2775.0	3048	2160
Crimes	2704.0	3384	1800
Transportations	2684.6	3408	1992
Environment	2654.4	3144	2184
Waste	2608.0	3024	2136

9. **Highlights the most engaged citizens and whether their voices are truly being addressed. Encourages recognition of "citizen champions."**

```
SELECT ct.citizen_id, ct.name, ct.surname,
    COUNT(i.issue_id) AS total_reports,
    ROUND(AVG(f.rating), 2) AS avg_rating_given,
```

```
    SUM(CASE WHEN i.current_status = 'resolved' THEN 1 ELSE 0 END) AS issues_resolved
FROM Citizen ct
JOIN Issue i ON ct.citizen_id = i.citizen_id
LEFT JOIN Feedback f ON i.issue_id = f.issue_id AND f.citizen_id = ct.citizen_id
GROUP BY ct.citizen_id, ct.name, ct.surname
ORDER BY total_reports DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

citizen_id	name	surname	total_reports	avg_rating_given	issues_resolved
32	Brooks	Spencer	6	NULL	4
33	Ophelie	Malanie	5	NULL	2
30	Keven	Gene	5	NULL	4
2	Josephine	Fosbraey	4	NULL	2
25	Devland	Carissa	4	NULL	2
8	Mikol	Wainwright	4	NULL	4
20	Ileana	Bradan	3	NULL	2
24	Dulcie	Jehu	3	NULL	3
31	Nannie	Chlo	3	NULL	1
12	Gladi	Karlen	3	NULL	1

10. **Government authorities to monitor office performance by issue category over the last year (helps identify which offices are efficient in resolving issues and which categories need attention)**

```
SELECT
    o.office_name,
    c.category_name,
    COUNT(i.issue_id) AS total_issues_reported,
    SUM(CASE WHEN i.current_status = 'resolved' THEN 1 ELSE 0 END) AS resolved_issues,
    AVG(TIMESTAMPDIFF(DAY, i.reported_at, i.completed_at)) AS avg_resolution_days,
    AVG(f.rating) AS avg_citizen_rating,
    COUNT(DISTINCT ci.citizen_id) AS unique_reporters,
    GROUP_CONCAT(DISTINCT CONCAT('Issue#', i.issue_id, ': ', i.description) ORDER BY
i.reported_at SEPARATOR '; ') AS recent_issues
FROM Issue i
```

JOIN Office o ON i.office_id = o.office_id

JOIN Category c ON i.category_id = c.category_id

LEFT JOIN Feedback f ON i.issue_id = f.issue_id

LEFT JOIN Citizen ci ON i.citizen_id = ci.citizen_id

WHERE i.reported_at >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR)

GROUP BY o.office_name, c.category_name

HAVING total_issues_reported > 0

ORDER BY resolved_issues DESC, avg_citizen_rating DESC, avg_resolution_days ASC;

```
SELECT
    o.office_name,
    c.category_name,
    COUNT(i.issue_id) AS total_issues_reported,
    SUM(CASE WHEN i.current_status = 'resolved' THEN 1 ELSE 0 END) AS resolved_issues,
    AVG(TIMESTAMPDIFF(DAY, i.reported_at, i.completed_at)) AS avg_resolution_days,
    AVG(f.rating) AS avg_citizen_rating,
    COUNT(DISTINCT ci.citizen_id) AS unique_reporters,
    GROUP_CONCAT(DISTINCT CONCAT('Issue#', i.issue_id, ': ', i.description) ORDER BY i.reported_at SEPARATOR '; ') AS recent_issues
FROM Issue i
JOIN Office o ON i.office_id = o.office_id
JOIN Category c ON i.category_id = c.category_id
LEFT JOIN Feedback f ON i.issue_id = f.issue_id
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

office_name	category_name	total_issues_reported	resolved_issues	avg_resolution_days	avg_citizen_rating	unique_reporters	recent_issues
Health Department	Environment	1	1	124.0000	1.0000	1	Issue#38: Mljdgi
Utilities Department	Disaster	1	1	123.0000	NULL	1	Issue#8: Daycug
Utilities Department	Waste	1	0	111.0000	2.0000	1	Issue#17: Ciakaz
Utilities Department	Transportations	1	0	117.0000	1.0000	1	Issue#32: Zugnre
Crimes Department	Environment	1	0	NULL	NULL	1	Issue#6: Duqsyw

11. **Average time to start handling issues per office**

SELECT o.office_name,

AVG(TIMESTAMPDIFF(DAY, i.reported_at, i.started_at)) AS avg_days_to_start

FROM Issue i

JOIN Office o ON o.office_id = i.office_id

WHERE i.started_at IS NOT NULL

GROUP BY o.office_id

ORDER BY avg_days_to_start;

```
SELECT o.office_name,
       AVG(TIMESTAMPDIFF(DAY, i.reported_at, i.started_at)) AS avg_days_to_start
FROM Issue i
JOIN Office o ON o.office_id = i.office_id
WHERE i.started_at IS NOT NULL
GROUP BY o.office_id
ORDER BY avg_days_to_start;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

office_name	avg_days_to_start
Waste Department	24.1667
Disaster Department	27.4000
Utilities Department	34.6000
Crimes Department	36.0000
Transportations Department	38.5000
Accident Department	40.7500
Environment Department	41.0000
Health Department	49.2857

12. Average time to resolve issues per office (from the time the issue was reported)

SELECT o.office_name,

AVG(TIMESTAMPDIFF(DAY, i.reported_at, i.completed_at)) AS avg_days_to_resolve

FROM Issue i

JOIN Office o ON o.office_id = i.office_id

WHERE i.completed_at IS NOT NULL

GROUP BY o.office_id

ORDER BY avg_days_to_resolve;

```
SELECT o.office_name,
       AVG(TIMESTAMPDIFF(DAY, i.reported_at, i.completed_at)) AS avg_days_to_resolve
FROM Issue i
JOIN Office o ON o.office_id = i.office_id
WHERE i.completed_at IS NOT NULL
GROUP BY o.office_id
ORDER BY avg_days_to_resolve;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

office_name	avg_days_to_resolve
Environment Department	106.7500
Transportations Department	107.8000
Crimes Department	111.2500
Disaster Department	116.2000
Utilities Department	116.2500
Accident Department	117.3333
Health Department	120.4286
Waste Department	135.3333

13. **Monthly trend of closed issues with average resolution time**

```
SELECT DATE_FORMAT(i.completed_at, '%Y-%m') AS month,
    COUNT(*) AS issues_completed,
    AVG(TIMESTAMPDIFF(DAY, i.started_at, i.completed_at)) AS avg_resolution_days
FROM Issue i
WHERE i.completed_at IS NOT NULL
GROUP BY month
ORDER BY month DESC;
```

```
19 •  SELECT DATE_FORMAT(i.completed_at, '%Y-%m') AS month,
20            COUNT(*) AS issues_completed,
21            AVG(TIMESTAMPDIFF(DAY, i.started_at, i.completed_at)) AS avg_resolution_days
22    FROM Issue i
23    WHERE i.completed_at IS NOT NULL
24    GROUP BY month
25    ORDER BY month DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

month	issues_completed	avg_resolution_days
2025-11	1	54.0000
2025-04	2	103.5000
2025-03	1	92.0000
2024-11	1	53.0000
2024-07	1	123.0000
2024-04	1	106.0000
2024-03	1	79.0000
2024-01	1	48.0000
2023-10	1	98.0000
2023-08	1	66.0000
2023-06	1	99.0000
2023-05	1	46.0000
2023-04	1	116.0000
2023-02	1	58.0000
2023-01	1	37.0000
2022-12	1	104.0000
2022-11	1	108.0000
2022-10	1	113.0000
2022-08	2	56.5000
2022-01	1	130.0000
2021-11	2	93.0000
2021-09	1	17.0000
2021-08	2	71.5000

14. **Update current_status of issue number 123 (transition ‘pending’→ ‘in_progress’).**

```
UPDATE Issue
SET current_status = 'in_progress'
WHERE issue_id = 123
```

AND current_status = 'pending';

```
27 •  UPDATE Issue
28    SET current_status = 'in_progress'
29    WHERE issue_id = 21
30      AND current_status = 'pending';
31
32
33
```

Output

Action Output

#	Time	Action	Message
2	12:41:18	UPDATE Issue SET current_status = 'in_progress' WHERE issue_id = 21 AND current_status = 'pending'	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

15. **Remove resolved issues, which are older than 3 years.**

```
DELETE FROM Issue
WHERE issue_id IN (
  SELECT issue_id
  FROM (
    SELECT issue_id
    FROM Issue
    WHERE current_status = 'resolved'
      AND completed_at < NOW() - INTERVAL 3 YEAR
  ) AS t
);
```

```
145 •  DELETE FROM Issue
146    WHERE issue_id IN (
147        SELECT issue_id
148        FROM (
149            SELECT issue_id
150            FROM Issue
151            WHERE current_status = 'resolved'
152              AND completed_at < NOW() - INTERVAL 3 YEAR
153        ) AS t
154    );
155
```

Output

Action Output

#	Time	Action	Message
1	13:15:47	DELETE FROM Issue WHERE issue_id IN (SELECT issue_id FROM (SELECT issue_id FROM Issue WHERE cur...	14 row(s) affected

16. **Top 5 Most Recent Citizens with Surnames Starting with 'A'**

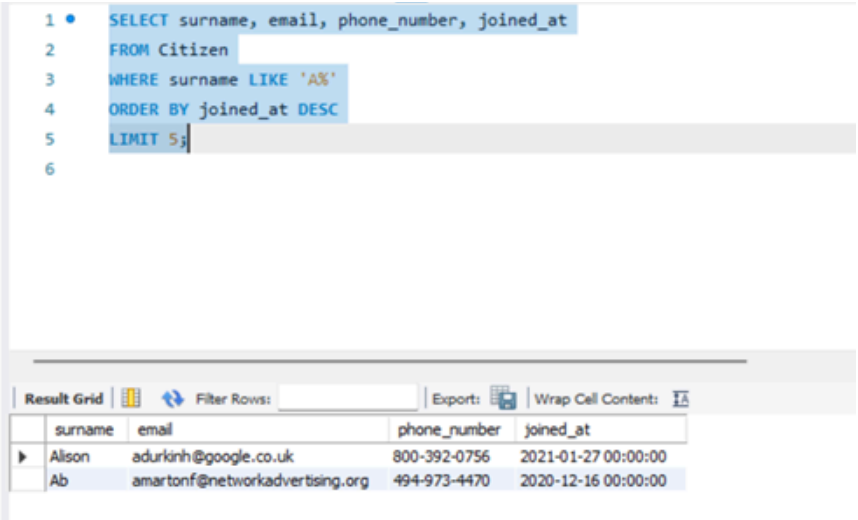
SELECT surname, email, phone_number, joined_at

FROM Citizen

WHERE surname LIKE 'A%'

ORDER BY joined_at DESC

LIMIT 5;



17. **Get the latest 5 feedback with citizen info**

SELECT f.feedback_id, f.comment, f.rating, f.created_at, c.surname, c.email

FROM Feedback f

JOIN Citizen c ON f.citizen_id = c.citizen_id

ORDER BY f.created_at DESC

LIMIT 5;

```sql
SELECT f.feedback_id, f.comment, f.rating, f.created_at, c.surname, c.email
FROM Feedback f
JOIN Citizen c ON f.citizen_id = c.citizen_id
ORDER BY f.created_at DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

feedback_id	comment	rating	created_at	surname	email
1	The road has been under construction for week...	1	2025-08-23 00:00:00	Welsh	wchesona@nba.com
5	Garbage collection service is inconsistent, some...	3	2025-07-10 00:00:00	Emalee	eadshedet@qq.com
19	Problem solved on the same day, excellent work!	5	2025-06-09 00:00:00	Karlen	kaiersa@mtv.com
20	Communication was poor, had to follow up multi...	4	2025-05-17 00:00:00	Carissa	ckobap@loc.gov
4	The problem was solved but took longer than e...	4	2025-04-17 00:00:00	Raoul	rlongstaffes@taobao.com

18. Media Records Sorted by Earliest Upload Date

SELECT media_id, url, caption, uploaded_at

FROM Media

ORDER BY uploaded_at ASC;

Query 1 | Limit to 1000 rows

```sql
SELECT media_id, url, caption, uploaded_at
FROM Media
ORDER BY uploaded_at ASC;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

media_id	url	caption	uploaded_at
1	https://imgs.example.com/transport/road-001.jpg	Pothole near school entrance #Transportations	2024-01-15 00:00:00
2	https://imgs.example.com/utilities/power-001.jpg	Fallen electric pole #Utilities	2024-03-03 00:00:00
3	https://imgs.example.com/waste/bin-001.jpg	Overflowing garbage bins #Waste	2024-06-18 00:00:00
4	https://imgs.example.com/crime/graffiti-001.jpg	Graffiti on community hall #Crimes	2024-08-27 00:00:00
5	https://imgs.example.com/health/clinic-001.jpg	Broken wheelchair ramp #Health	2024-09-12 00:00:00
6	https://imgs.example.com/accident/sign-001.jpg	Damaged stop sign #Accident	2024-10-30 00:00:00
7	https://imgs.example.com/env/tree-001.jpg	Fallen tree blocking road #Environment	2024-11-19 00:00:00
8	https://imgs.example.com/disaster/flood-001.jpg	Street flooded after storm #Disaster	2024-12-05 00:00:00
9	https://imgs.example.com/transport/busstop-0...	Broken bench at bus stop #Transportations	2025-01-14 00:00:00
10	https://imgs.example.com/utilities/water-001.jpg	Water pipe leakage #Utilities	2025-02-02 00:00:00
11	https://imgs.example.com/waste/dump-001.jpg	Illegal dumping in alley #Waste	2025-02-25 00:00:00
12	https://imgs.example.com/crime/light-001.jpg	Streetlight not working #Crimes	2025-03-08 00:00:00

Media 8

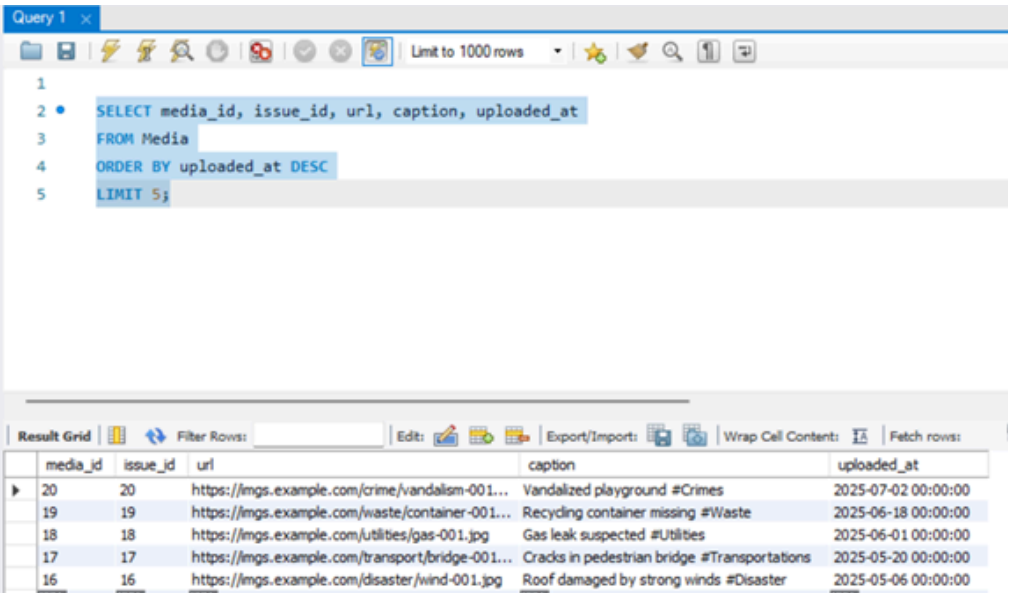
19. **Retrieve the 5 most recent media entries along with their issue ID and caption:**

```
SELECT media_id, issue_id, url, caption, uploaded_at

FROM Media

ORDER BY uploaded_at DESC

LIMIT 5;
```



media_id	issue_id	url	caption	uploaded_at
20	20	https://imgs.example.com/crime/vandalism-001...	Vandalized playground #Crimes	2025-07-02 00:00:00
19	19	https://imgs.example.com/waste/container-001...	Recycling container missing #Waste	2025-06-18 00:00:00
18	18	https://imgs.example.com/utilities/gas-001.jpg	Gas leak suspected #Utilities	2025-06-01 00:00:00
17	17	https://imgs.example.com/transport/bridge-001...	Cracks in pedestrian bridge #Transportations	2025-05-20 00:00:00
16	16	https://imgs.example.com/disaster/wind-001.jpg	Roof damaged by strong winds #Disaster	2025-05-06 00:00:00

20. **Find Issues Assigned to Each Office that Are Still Pending**

```
SELECT

        o.office_name,

        COUNT(i.issue_id) AS pending_issues

FROM Issue i

JOIN Office o ON i.office_id = o.office_id

WHERE i.current_status = 'pending'

GROUP BY o.office_name

ORDER BY pending_issues DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

office_name	pending_issues
Accident Department	2
Health Department	2
Utilities Department	2
Crimes Department	1
Transportations Department	1
Disaster Department	1

Result 1